100 AI Deployment Challenges

Fine-tuning, GPU, Inference & Infrastructure — Questions with Short Answers

Fine-tuning Challenges

1. **What is the biggest challenge in fine-tuning large models?**
   A: Requires large GPU memory and careful hyperparameter tuning to avoid overfitting or catastrophic forgetting.

2. **Why does fine-tuning cause catastrophic forgetting?**
   A: The model overwrites previously learned general knowledge while adapting too aggressively to new data.

3. **How do you reduce GPU memory needs during fine-tuning?**
   A: Use parameter-efficient methods like LoRA, QLoRA, or adapters instead of full fine-tuning.

4. **What is LoRA and why is it useful?**
   A: Low-Rank Adaptation trains small added matrices instead of all weights, cutting memory and compute cost.

5. **What is QLoRA?**
   A: A technique combining 4-bit quantization with LoRA to fine-tune large models on consumer-grade GPUs.

6. **How do you prevent overfitting during fine-tuning on small datasets?**
   A: Use early stopping, dropout, regularization, and data augmentation.

7. **What's a common mistake when fine-tuning on domain data?**
   A: Using too high a learning rate, which destroys pre-trained knowledge.

8. **How do you choose the right fine-tuning learning rate?**
   A: Start very low (e.g., 1e-5 to 5e-5) and use a warmup + decay schedule.

9. **Why is data quality more important than data quantity in fine-tuning?**
   A: A small set of high-quality, well-labeled examples improves performance more than large noisy data.

10. **What challenge arises from fine-tuning on biased data?**
    A: The model inherits and can amplify biases present in the training set.

11. **How do you validate a fine-tuned model before deployment?**
    A: Run held-out evaluation sets, human review, and regression tests against the base model.

12. **What is the risk of full fine-tuning vs. parameter-efficient tuning?**
    A: Full fine-tuning needs much more GPU memory/storage and risks overfitting on small datasets.

13. **How do you fine-tune efficiently across multiple tasks?**
    A: Use multi-task learning or modular adapters per task to avoid interference.

14. **What challenge exists in fine-tuning instruction-following models?**
    A: Balancing helpfulness with safety so the model doesn't become unsafe or overly compliant.

15. **How do you track fine-tuning experiments effectively?**
    A: Use experiment tracking tools like MLflow or Weights & Biases to log configs, metrics, and checkpoints.

GPU & Hardware Challenges

16. What is the most common GPU bottleneck during training?

A: Memory (VRAM) limitations, especially with large batch sizes or long sequences.

17. How do you handle GPU out-of-memory (OOM) errors?

A: Reduce batch size, use gradient accumulation, mixed precision, or gradient checkpointing.

18. What is gradient checkpointing?

A: A technique that trades compute for memory by recomputing activations during backprop instead of storing them.

19. What is mixed precision training?

A: Using FP16/BF16 instead of FP32 to reduce memory usage and speed up training with minimal accuracy loss.

20. How do you scale training across multiple GPUs?

A: Use data parallelism, model parallelism, or frameworks like DeepSpeed/FSDP.

21. What is the challenge with multi-GPU communication overhead?

A: Synchronizing gradients across GPUs can become a bottleneck, especially over slow interconnects.

22. How do you choose between data parallelism and model parallelism?

A: Data parallelism for models that fit in one GPU; model parallelism when the model is too large for one GPU.

23. What is GPU fragmentation and why does it matter?

A: Memory gets split into unusable chunks over time, reducing effective available memory.

24. How do you reduce GPU idle time during training?

A: Optimize data loading pipelines and use prefetching/asynchronous data loaders.

25. What challenge comes with GPU cost in production?

A: Balancing inference latency/throughput requirements against the high cost of GPU instances.

26. How do you decide between cloud GPUs and on-premise GPUs?

A: Consider workload predictability, cost over time, scaling needs, and data security requirements.

27. What is GPU quantization and why use it for deployment?

A: Reducing model weight precision (e.g., INT8) to shrink memory footprint and speed up inference.

28. What's a common issue with multi-tenant GPU clusters?

A: Resource contention and unfair scheduling between teams or jobs.

29. How do you monitor GPU utilization in production?

A: Use tools like NVIDIA DCGM, Prometheus exporters, or cloud-native GPU monitoring dashboards.

30. What challenge arises from GPU driver/CUDA version mismatches?

A: Compatibility issues that cause crashes or silent performance degradation between library versions.

Model Serving & Inference Challenges

31. What is the main challenge in real-time model serving?

A: Achieving low latency while maintaining high throughput under variable traffic loads.

32. How do you reduce inference latency for large models?

A: Use model quantization, distillation, caching, and optimized inference engines like TensorRT or vLLM.

33. What is dynamic batching and why is it used?

A: Grouping multiple inference requests together to better utilize GPU compute and improve throughput.

34. What challenge does autoscaling for ML inference face?

A: Cold-start latency when scaling up new instances that need to load large models.

35. How do you handle cold-start problems in serverless ML deployment?

A: Keep warm instances, use lighter models, or pre-load model weights into memory caches.

36. What is the risk of using a single large model for all requests?

A: Inefficient resource usage for simple requests that don't need full model capacity.

37. How do you solve this inefficiency?

A: Use model cascading or routing — small models handle easy cases, large models handle complex ones.

38. What challenge exists with streaming responses from LLMs?

A: Maintaining low time-to-first-token while still ensuring consistent output quality.

39. How do you manage concurrent requests to a single GPU-hosted model?

A: Use request queuing, batching, and concurrency limits to avoid memory overload.

40. What's a key challenge in deploying multiple model versions simultaneously?

A: Managing routing, resource allocation, and consistent monitoring across versions.

41. How do you handle inference failures gracefully?

A: Add retries, fallback models, circuit breakers, and clear error responses.

42. What's the challenge with caching LLM responses?

A: Balancing cache hit rate against staleness and ensuring sensitive data isn't cached improperly.

43. How do you reduce cost of serving LLMs at scale?

A: Use smaller distilled models, quantization, batching, and tiered model routing.

44. What challenge comes with multi-modal model deployment?

A: Handling different data types (text, image, audio) with different latency and resource profiles.

45. How do you ensure consistent output format from deployed LLMs?

A: Use structured output constraints, function calling, or output validation/parsing layers.

Infrastructure & Scalability Challenges

46. What's a common challenge in scaling ML infrastructure?

A: Balancing elasticity for variable demand against the high fixed cost of GPU resources.

47. How do you handle distributed training failures?

A: Implement checkpointing so jobs can resume from the last saved state instead of restarting.

48. What is the challenge with container orchestration for ML workloads?

A: Managing GPU scheduling, resource quotas, and node affinity in Kubernetes-like systems.

49. How do you manage storage for large model checkpoints?

A: Use efficient formats (e.g., safetensors), incremental checkpointing, and tiered storage.

50. What challenge exists with network bandwidth in distributed training?

A: Slow interconnects can bottleneck gradient synchronization across nodes.

51. How do you address this network bottleneck?

A: Use high-speed interconnects (NVLink, InfiniBand) and gradient compression techniques.

52. What is the challenge of multi-cloud GPU deployment?

A: Inconsistent APIs, pricing, and availability across providers complicate orchestration.

53. How do you plan capacity for unpredictable AI workloads?

A: Use autoscaling combined with reserved baseline capacity and load forecasting.

54. What challenge arises with model artifact versioning?

A: Ensuring reproducibility while tracking which data, code, and weights produced each version.

55. How do you handle dependency conflicts across ML environments?

A: Use containerization (Docker) and environment management tools like conda or pip-tools.

56. What's the risk of vendor lock-in with managed AI platforms?

A: Difficult migration and pricing dependency if you build deeply on one provider's proprietary stack.

57. How do you mitigate vendor lock-in risk?

A: Use open standards (ONNX, ML metadata) and abstract infrastructure behind portable interfaces.

58. What challenge comes with edge deployment of AI models?

A: Limited compute/memory on edge devices requires aggressive model compression.

59. How do you compress models for edge deployment?

A: Use pruning, quantization, and knowledge distillation to shrink model size.

60. What is a common challenge with hybrid cloud-edge AI systems?

A: Synchronizing models and data consistently between edge devices and central cloud.

Monitoring, Reliability & Maintenance Challenges

61. What is the biggest challenge in monitoring deployed models?

A: Detecting silent performance degradation (model or data drift) before it impacts users.

62. How do you detect model drift in production?

A: Continuously track input distribution shifts and output quality metrics against a baseline.

63. What challenge exists in setting up alerting for ML systems?

A: Avoiding alert fatigue while still catching real anomalies in latency, errors, or accuracy.

64. How do you handle rollback when a new model underperforms?

A: Maintain blue-green or canary deployments to switch back to the previous stable version quickly.

65. What is the challenge of A/B testing ML models in production?

A: Ensuring statistically valid results without harming user experience during the test.

66. How do you ensure reproducibility of production model behavior?

A: Pin software/library versions, fix random seeds, and log exact configs used at inference.

67. What challenge comes with logging LLM outputs for auditing?

A: Balancing thorough logging with user privacy and storage cost constraints.

68. How do you handle compliance audits for deployed AI models?
A: Maintain documentation of training data, evaluation results, and decision logic (model cards).

69. What's a common challenge in maintaining model freshness?
A: Deciding how often to retrain without incurring excessive cost or instability.

70. How do you trigger retraining effectively?
A: Use automated drift detection thresholds or scheduled retraining cadences.

71. What challenge exists with regression testing for ML updates?
A: Subtle behavior changes can degrade performance on edge cases not covered by standard tests.

72. How do you build regression test suites for ML models?
A: Maintain a curated set of critical test cases and golden outputs to compare against new versions.

73. What is the risk of insufficient observability in ML pipelines?
A: Failures go undetected until they cause major business impact downstream.

74. How do you improve observability of ML pipelines?
A: Add tracing, structured logging, and metrics at every pipeline stage (data, training, serving).

75. What challenge arises from supporting legacy deployed models?
A: Maintaining old dependencies and infrastructure while still innovating on new systems.

Cost, Security & Governance Challenges

76. What is the main cost challenge in deploying LLMs?
A: Balancing inference cost per request against required latency and accuracy.

77. How do you reduce overall LLM deployment costs?
A: Use smaller/distilled models, caching, batching, and spot/reserved GPU instances.

78. What security challenge is common in AI deployment?
A: Protecting model weights and APIs from unauthorized extraction or misuse.

79. How do you prevent model extraction attacks?
A: Rate-limit APIs, monitor for suspicious query patterns, and use watermarking techniques.

80. What challenge exists with prompt injection in deployed LLM apps?
A: Malicious inputs can manipulate the model into ignoring intended instructions.

81. How do you mitigate prompt injection risks?
A: Use input sanitization, system prompt isolation, and output validation layers.

82. What is the governance challenge with deploying AI at scale?
A: Ensuring consistent policy enforcement (bias, safety, privacy) across many deployed models.

83. How do you ensure data privacy during inference?
A: Avoid logging sensitive inputs, use encryption in transit/at rest, and apply data minimization.

84. What challenge comes with multi-tenant SaaS AI deployment?
A: Isolating customer data and ensuring one tenant's usage doesn't affect another's performance.

85. How do you handle access control for fine-tuned proprietary models?

A: Use API key scoping, role-based access control, and audit logging.

86. What challenge exists in estimating ROI for AI deployment?

A: Quantifying business impact against ongoing GPU and maintenance costs is hard to predict upfront.

87. How do you control runaway cloud GPU spending?

A: Set budget alerts, auto-shutdown idle instances, and use spot instances for non-critical jobs.

88. What challenge exists with compliance for cross-border data in AI deployment?

A: Different regions have different data residency and processing laws that must be respected.

89. How do you secure model endpoints from DDoS-style abuse?

A: Apply rate limiting, authentication, and traffic anomaly detection on the API gateway.

90. What governance challenge arises with third-party fine-tuned models?

A: Verifying the provenance and safety of models trained outside your organization.

Team, Process & Scaling Operations

91. What challenge do teams face moving from notebook experiments to production?

A: Refactoring research code into reliable, tested, and maintainable production pipelines.

92. How do you reduce friction between data science and DevOps teams?

A: Adopt MLOps practices and shared tooling so both teams work from the same pipelines.

93. What challenge exists in standardizing deployment across teams?

A: Different teams use different frameworks/tools, making centralized support difficult.

94. How do you solve deployment standardization issues?

A: Build internal platform tooling/templates that enforce consistent deployment patterns.

95. What is a common challenge when scaling a small AI team's deployment process?

A: Manual deployment steps that don't scale and introduce human error as volume grows.

96. How do you reduce manual deployment errors?

A: Automate via CI/CD pipelines with built-in testing, validation, and approval gates.

97. What challenge exists in keeping documentation updated for evolving models?

A: Fast iteration cycles mean docs quickly go stale without dedicated process ownership.

98. How do you keep deployment documentation current?

A: Tie documentation updates to the deployment pipeline as a required release step.

99. What challenge exists in coordinating release schedules for interdependent models?

A: Updating one model can break downstream models or pipelines that depend on its outputs.

100. How do you manage interdependent model releases safely?

A: Use versioned APIs/contracts and staged rollouts to avoid breaking downstream consumers.

101. What is the overarching challenge in AI deployment as a whole?

A: Continuously balancing speed, cost, reliability, and safety as systems and scale evolve.